UML and Code Kit

K For the moment you know the pattern but freeze on the boxes, names, arrows and code. Use it alongside the main note.

KIT
CONTENTS

K Method

UML and Code Kit 1
Why you lose track, and the fix 2
Step 1: getting the names out of the question 3
Step 2: the box and the arrow, decided by questions 4
Step 3: writing the code, in order 5

01 Singleton

Singleton: fill-in kit 6

02 Factory

Factory: fill-in kit 7

03 Facade

Facade: fill-in kit 8

04 Adapter

Adapter: fill-in kit 9

05 Proxy

Proxy: fill-in kit 10

06 Decorator

Decorator: fill-in kit 11

07 Template

Template Method: fill-in kit 12

08 Chain

Chain of Responsibility: fill-in kit 13

09 State

State: fill-in kit 14

P Practice

Two patterns in one question, and if you freeze 15
Practice: three new scenarios 16
Blank slot sheet 17

How this kit is different from the note: the note explains each pattern with one scenario. This kit gives you the **empty shape** of each pattern, with the role names on the boxes, and tells you **where in any question to find the word that goes in each box**. You fill in the shape; you never have to invent it.

Where boxes go on the page

1. Interface / abstract class: **top middle**.
2. Concrete classes: **one row underneath**, triangles pointing up into the parent.
3. The pattern's special box (decorator, adapter, proxy, context, factory): **to the side**, with its has-a arrow going sideways.
4. Client: **far left**, one arrow to what it uses.

The one special link per pattern

Singleton	class → itself (`instance`)
Factory	factory ⇢ each product «create»
Facade	facade → every subsystem
Adapter	adapter → adaptee
Proxy	proxy → real subject
Decorator	decorator ◇→ component
Template	none: just triangles
Chain	handler ◇→ itself (`next`)
State	context ◇→ state interface

Why you lose track, and the fix

K You already pick the right pattern. What breaks is doing four jobs in your head at the same time.

METHOD
READ FIRST

The four jobs you are juggling

JOB 1	JOB 2	JOB 3	JOB 4
Naming	**Shape**	**Symbols**	**Amount**
What do I call each class, field and method?	Which boxes does this pattern have, and which one is on top?	Which arrow, which end, dashed or solid, + or −?	How much code, which classes, where do I stop?

Doing all four while drawing is what makes you lose the thread: you invent a name, forget the arrow, change the name in the code, and the diagram and code stop matching. **The fix is to do them one at a time, in a fixed order, and copy jobs 2 and 3 from this kit instead of remembering them.**

The routine for every diagram or code question

same 4 steps, any pattern

STEP	TIME	WHAT YOU DO
1 · Slot table	2 min	Open this kit at the pattern's page. In your rough-work space, copy the **Role** column. Next to each role, write the word from the question (the page tells you where to look). **This little table is now the only place names come from.**
2 · Draw	4–6 min	Copy the pattern's skeleton diagram box by box, writing **your** name from the slot table instead of the role name. The arrows are already decided by the skeleton, so just copy them.
3 · Code	5–8 min	Write the classes in the order of the slot table, top to bottom. Use the code skeleton on the pattern page; change only the names.
4 · Check	1 min	Run the checklist at the bottom of the pattern page. Most lost marks are on that list.

> The golden rule: `diagram and code copy their names from the same slot table`, so they always match, and you never have to remember a name halfway through.

How much to draw

by marks

3–4 m	The pattern's core boxes only (interface + 2 concrete + the special box). Names and arrows correct; 1–2 methods each.
5–6 m	Full skeleton from the pattern page, with fields and 2–3 methods in each box, plus the client.
8–10 m	Full skeleton + every concrete class the question lists + stereotypes («interface», «abstract») + multiplicities on has-a links + a 1–2 line note under the diagram saying which box plays which role.

How much code

by marks

2–3 m	Exactly what is asked (often just the interface or one class). Nothing extra.
4–5 m	What is asked, in full (fields, constructor, methods), plus 3 lines of client code showing it used.
6+ m	Interface/abstract class + the pattern class + two concrete classes + client with a comment showing the output.

Never needed unless asked: imports, getters/setters, `main`, full error handling.

Two methods are enough

For each interface, pick **the one or two actions the question is about**. A pricing question → `getCost()`, `getDescription()`. A states question → one method per event. An approval question → `handle(request)`. Don't list every possible method; examiners mark the pattern, not completeness.

Step 1: getting the names out of the question

K Every name you need is already in the question. You only have to sort the words into the right slots.

METHOD
NAMING

Six sorting rules

WORD IN THE QUESTION	BECOMES	EXAMPLE
1. The **general word** for the thing ("a payment method", "a book", "an account", "a state")	the **interface** or **abstract class**	"different payment methods" → `PaymentMethod`
2. The **listed kinds** ("credit card, PayPal, cash on delivery")	**concrete classes**, one each	`CreditCardPayment`, `PayPalPayment`, `CashOnDeliveryPayment`
3. Actions (verbs: validate, scan, approve, calculate)	**methods**: verb + noun	"process the payment" → `processPayment()`
4. Data the thing has (price, name, balance, days)	**fields**	`- price : double`
5. Who uses it (the system, the app, the kiosk, the desk)	the **client** class	`CheckoutSystem`, `LibraryApp`
6. A name or method **written in the question**	**copy it exactly**, same letters and case	2024: "`getCost()` and `getDescription()`"

No word for a pattern box? Use the general word + the pattern's suffix: `…Decorator`, `…Adapter`, `…Proxy`, `…Factory`, `…Facade`, `…State`, `…Handler`. Example: the general word is Burger → `BurgerDecorator`.

How to write names

Class / interface	Noun, capital first, joined words: `GroceryItem`
Method	verb first, small first letter: `getCost()`, `approveLeave()`
Field	noun, small first letter: `price`, `nextHandler`
Constant	capitals: `MAX_PLAYERS`, `TAX_RATE`
Singular	`Book`, not `Books`; one class = one thing

Worked example: sorting a new paragraph

QUESTION (NOT FROM A PAST PAPER)

"A burger shop lets customers order a **basic burger** and add optional **extras** such as **cheese**, **bacon** and **large size**. The **price** and the **order description** must be calculated for any combination without creating subclasses for every combination."

Pattern: "optional extras… without subclasses for every combination" → Decorator (you can already do this part).

Sorting: general word = burger → `Burger` interface · base = basic burger → `BasicBurger` · listed extras → `Cheese`, `Bacon`, `LargeSize` · no word for the decorator box → suffix rule → `BurgerDecorator` · actions = calculate price, describe order → `getCost()`, `getDescription()`.

ROLE (FROM THE KIT)	MY WORD
Component	`Burger`
ConcreteComponent	`BasicBurger`
Decorator	`BurgerDecorator`
ConcreteDecorators	`Cheese, Bacon, LargeSize`
operations	`getCost(), getDescription()`

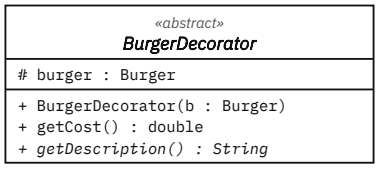
That five-line table is all you need. Now open the Decorator page and copy the shape with these words.

K Step 2: the box and the arrow, decided by questions

You don't need to remember UML. Answer the yes/no questions and the symbol picks itself.

METHOD
DRAWING

What goes inside one box



① name (italic = abstract)

② fields: – name : Type

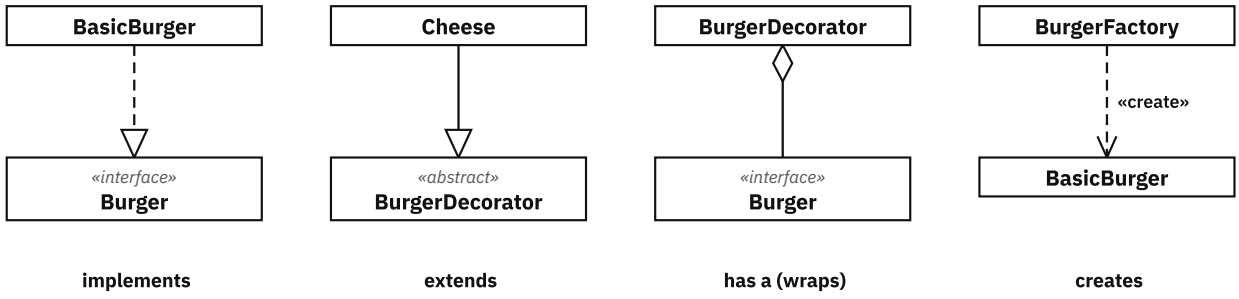
③ methods: + name(p : Type) : Return

Box rules

- **Top:** the name. Write «interface» above it for an interface. Write the name in *italics* (or «abstract» above it) for an abstract class.
- **Middle:** fields, `- name : Type`. Only the 1–3 the pattern needs (the wrapped object, the next handler, the current state, the price).
- **Bottom:** methods, `+ name(param : Type) : ReturnType`. 2–3 is enough.
- **Interface:** leave the middle box empty (interfaces have no fields).
- `+` public · `–` private · `#` protected · underline = static · *italic method* = abstract

Which arrow? Ask in this order and stop at the first "yes"

QUESTION ABOUT CLASS A AND CLASS B	DRAW	EXAMPLE
1. Does A say `implements B`? (B is an interface)	**dashed line, hollow triangle at B**	`BasicBurger` → `Burger`
2. Does A say `extends B`? (B is a class / abstract class)	**solid line, hollow triangle at B**	`Cheese` → `BurgerDecorator`
3. Does A have a **field** of type B? (A "has a" B)	**solid line, open arrow at B**; add a hollow diamond at A if A wraps or collects B	decorator ◇→ component, context ◇→ state, proxy → real subject
4. Does A only **use** or **create** B inside a method?	**dashed line, open arrow at B**; label «create» for factories	factory ⇢ product; client ⇢ `getInstance()`



Memory trick: triangle = "is a" (it's a kind of that), and the triangle always touches the **parent**. Dashed triangle = interface parent, solid triangle = class parent. A plain arrow = "has a" or "uses". Note: a decorator has **both** a triangle (it is a Component) and a diamond arrow (it holds a Component). That's correct, not a mistake.

Step 3: writing the code, in order

K Every pattern's code is built from the same seven small moves. Learn the moves once; the pattern pages just arrange them.

METHOD
CODE

Write the classes in this order

same order as your slot table

1	2	3	4	5
Interface / abstract	**One concrete class**	**The pattern class**	**More concretes**	**Client**
The general word. Method headers only (or the final template method).	Fields, constructor, every method. In full.	Decorator, adapter, proxy, factory, context, handler.	Only if marks allow. Same shape as #2.	3–5 lines that use it, with `// output`.

Put a comment with the role above each class (`// COMPONENT`, `// ADAPTER`). It shows the examiner you know the parts, and it keeps **you** on track.

The seven moves

```
// MOVE 1: an interface = method headers
public interface Burger {
  double getCost();
  String getDescription();
}

// MOVE 2: implements = fill in every method
public class BasicBurger implements Burger {
  public double getCost() { return 500; }
  public String getDescription() {
    return "Basic burger"; }
}

// MOVE 3: has-a = field + constructor
public abstract class BurgerDecorator
    implements Burger {
  protected final Burger burger;   // field
  public BurgerDecorator(Burger b) {
    burger = b; }                  // set it
}

// MOVE 4: delegate = call the field, add
public double getCost() {
  return burger.getCost() + 150; }
```

```
// MOVE 5: extends + super(...)
public class Cheese extends BurgerDecorator {
  public Cheese(Burger b) { super(b); }
  // ... override getCost() with MOVE 4
}

// MOVE 6: a fixed order you can't change
public abstract class Process {
  public final void run() {   // final!
    stepOne(); stepTwo(); }
  protected abstract void stepTwo();
  private void stepOne() { }
}

// MOVE 7: only one object
public final class Logger {
  private static Logger instance;
  private Logger() { }        // private!
  public static synchronized Logger
      getInstance() {
    if (instance == null)
      instance = new Logger();
    return instance; }
}
```

Which moves each pattern uses

Singleton	7	**Decorator**	1, 2, 3, 4, 5
Factory	1, 2 + a `switch` that returns `new …`	**Template**	6, 5
Facade	3 (several fields), 4 (call them in order)	**Chain**	3 (field `next`), 4 (pass to next), 5
Adapter	1, 2, 3, 4 (call the adaptee's method)	**State**	1, 2, 3 (field `state`), 4 (`state.event(this)`)
Proxy	1, 2, 3, 4 (check first, then call)		

01 Singleton: fill-in kit

One box. The hard part is only the three fixed ingredients, which never change names.

KIT
SINGLETON

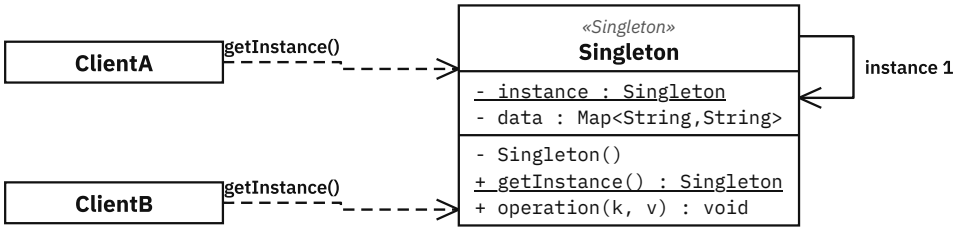
Slot table

example: "the hospital must use a single appointment scheduler"

ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
Singleton class	the thing there must be "only one" / "a single global" of	`AppointmentScheduler`
instance	fixed name, always `instance` (static)	`instance`
getInstance()	fixed name, always `getInstance()` (static)	`getInstance()`
shared data	what must be the same for everyone (queue, counter, rate, file)	`bookings : Map<String,String>`
operation()	what the one object does (verb in the question)	`book(patient, time)`
Clients	the parts that use it ("all counters", "every department")	`ReceptionDesk`, `DoctorPortal`

Skeleton diagram

box names = roles; replace each with the word in the slot table's Example column




```
// SINGLETON (rename: Singleton -> your class)
public final class Singleton {
  private static Singleton instance;  // one copy
  private final Map<String, String> data =
      new HashMap<>();                // shared

  private Singleton() { }             // no "new"

  public static synchronized Singleton
      getInstance() {
    if (instance == null)
      instance = new Singleton();
    return instance;
  }
  public synchronized void operation(String k,
                                     String v) {
    data.put(k, v); }
}
// CLIENT: never "new", always getInstance()
Singleton s = Singleton.getInstance();
s.operation("P01", "10:30");
```

"Maximum N" instead of one?

That's a **Multiton**: same three ingredients, but the static field is a `List` or `Map`, and the static method refuses (or reuses) once it holds N objects. Note tab 01 has the cricket version.

Check before moving on

- ☐ constructor **private**
- ☐ `instance` **static** and underlined in the diagram
- ☐ `getInstance()` **static**, underlined
- ☐ clients call `getInstance()`, never `new`
- ☐ said "thread-safe" (synchronized) in the text

02 Factory: fill-in kit

One interface, one class per listed kind, one factory with a switch. The listed kinds are always in the question.

KIT
FACTORY

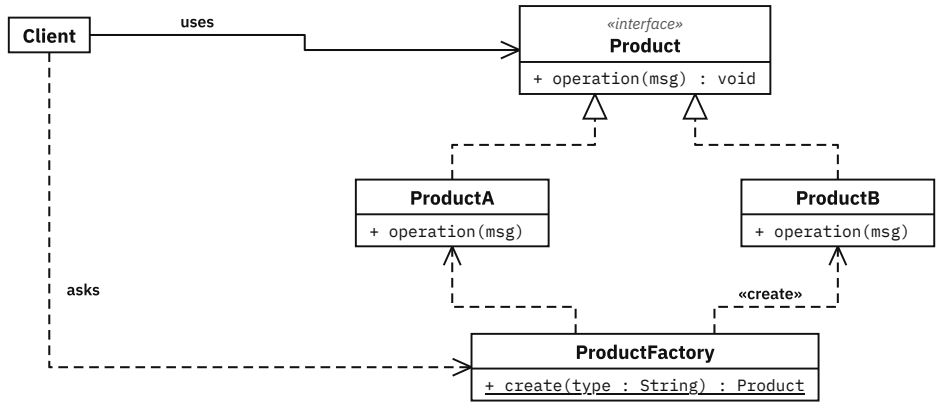
Slot table

example: "alerts are sent as email, SMS or push depending on the user's choice"

ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
Product «interface»	the general word before the list ("types of X", "different X")	`Notification`
ConcreteProducts	the listed kinds, one class each	`EmailNotification`, `SmsNotification`, `PushNotification`
operation()	the action every kind does	`send(message)`
Factory	general word + `Factory`	`NotificationFactory`
create(type)	`create` + general word, or the name the question gives	`create(type) : Notification`

Skeleton diagram

dashed triangles up to the interface; dashed «create» arrows from the factory




```
// PRODUCT
public interface Product {
  void operation(String msg);
}
// CONCRETE PRODUCT (one per listed kind)
public class ProductA implements Product {
  public void operation(String msg) {
    System.out.println("A: " + msg); }
}
public class ProductB implements Product {
  public void operation(String msg) {
    System.out.println("B: " + msg); }
}
```

```
// FACTORY: the only place with "new"
public class ProductFactory {
  public static Product create(String type) {
    switch (type) {
      case "A": return new ProductA();
      case "B": return new ProductB();
      default: throw new
          IllegalArgumentException(type);
    }
  }
}
// CLIENT: only knows the interface
Product p = ProductFactory.create("A");
p.operation("Hello");    // A: Hello
```

Question says "Factory Method" / "subclasses decide"?

Use the GoF form instead: an abstract `Creator` with `public abstract Product createProduct();` and one subclass per kind (`CreatorA` returns `new ProductA()`). Diagram: note tab 02, "The strict GoF version".

Check

- ☐ factory **returns the interface type**, not a concrete class
- ☐ client has no `new ProductA()`
- ☐ «create» arrows dashed, from the factory
- ☐ one concrete class per listed kind

03 Facade: fill-in kit

One front class with one big method. Behind it, one box per service the question lists.

KIT
FACADE

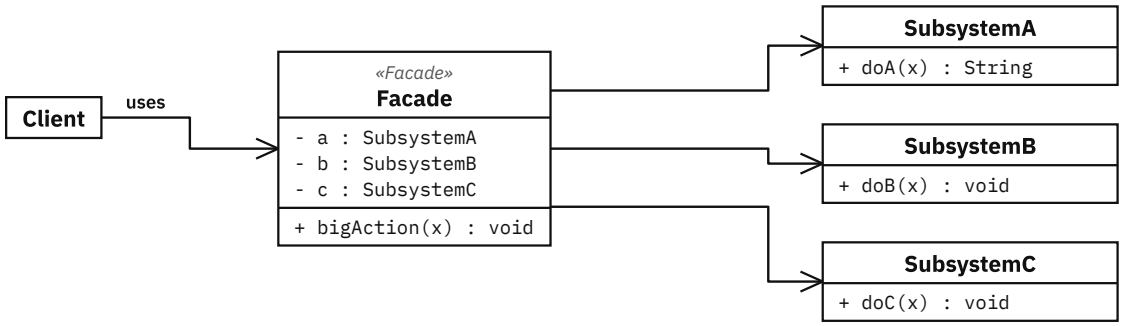
Slot table

example: "one check-in action that internally assigns a room, takes a deposit and issues a key card"

ROLE	FIND IT IN THE QUESTION BY...	EXAMPLE
Facade	the name of the whole job + `Facade`	`CheckInFacade`
facade method	the one action the user wants ("one simple interface for...")	`checkIn(guest, roomType)`
Subsystems	each service listed, usually after "internally calling" / "combine"	`RoomService`, `BillingService`, `KeyCardService`
their methods	what each service does, one verb each	`assignRoom()`, `chargeDeposit()`, `issueKey()`
Client	the screen / app the user touches	`FrontDeskApp`

Skeleton diagram

arrows go from the facade to the subsystems, never back




```
// SUBSYSTEMS: do the real work
class SubsystemA {
  String doA(String x) {
    return "A done for " + x; } }
class SubsystemB {
  void doB(String x) {
    System.out.println("B done"); } }
class SubsystemC {
  void doC(String x) {
    System.out.println("C done"); } }

// CLIENT: one call
Facade f = new Facade();
f.bigAction("Guest01");
```

```
// FACADE: calls them in the right order
public class Facade {
  private final SubsystemA a = new SubsystemA();
  private final SubsystemB b = new SubsystemB();
  private final SubsystemC c = new SubsystemC();

  public void bigAction(String x) {
    String r = a.doA(x);   // 1st
    b.doB(r);              // 2nd
    c.doC(r);              // 3rd
  }
}
```

Check

☐ one facade method that does the whole job ☐ one field per subsystem ☐ arrows only facade → subsystems ☐ no business rules inside the facade (it only calls) ☐ client talks only to the facade

04 Adapter: fill-in kit

Two method names are in the question: the old one your system calls, and the new one the vendor class has. The adapter joins them.

KIT
ADAPTER

Slot table

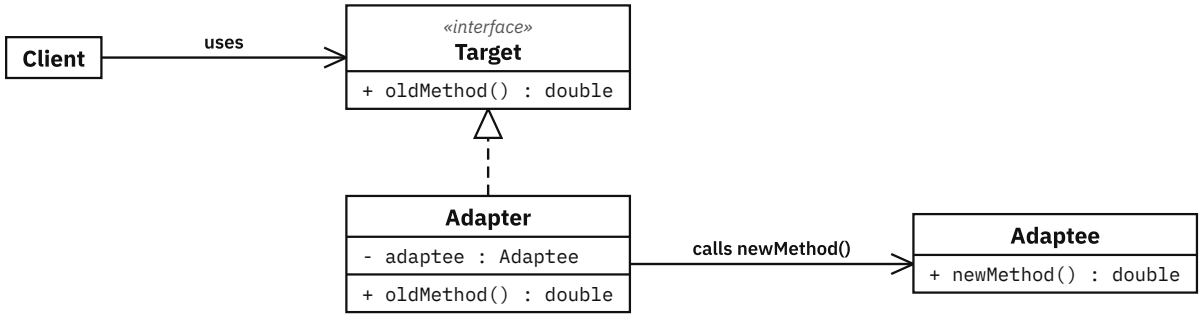
example: "the app calls getTemperature(); the new vendor sensor only has readCelsius()"

ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
Target «interface»	what your system **already calls** (the old method) → make it an interface	`Thermometer` with `getTemperature()`
Adaptee	the **new / legacy / vendor** class you can't change, with its **different** method	`VendorSensor` with `readCelsius()`
Adapter	adaptee name + `Adapter`	`VendorSensorAdapter`
Client	the existing system with the "hundreds of calls"	`WeatherApp`

Test: **Target method = old name. Adaptee method = new name.** Inside the adapter, the old name calls the new name.

Skeleton diagram

adapter implements the target and holds the adaptee




```java
// TARGET: what the client already uses
public interface Target {
  double oldMethod();
}
// ADAPTEE: vendor class, can't change it
public class Adaptee {
  public double newMethod() {
    return 24.5; }
}
// CLIENT: only the creation line changes
Target t = new Adapter(new Adaptee());
double v = t.oldMethod(); // old calls stay
```

```java
// ADAPTER: is a Target, has an Adaptee
public class Adapter implements Target {
  private final Adaptee adaptee;
  public Adapter(Adaptee a) {
    this.adaptee = a; }

  public double oldMethod() {
    // translate: old call -> new call
    // (convert units / format here too)
    return adaptee.newMethod();
  }
}
```

Check

☐ adapter **implements Target** (dashed triangle) ☐ adapter **has an Adaptee field** (solid arrow) ☐ adaptee untouched ☐ the old method's body calls the new method ☐ named all four participants in words

05 Proxy: fill-in kit

Two classes with the same interface. The proxy checks or delays first, then calls the real one.

KIT
PROXY

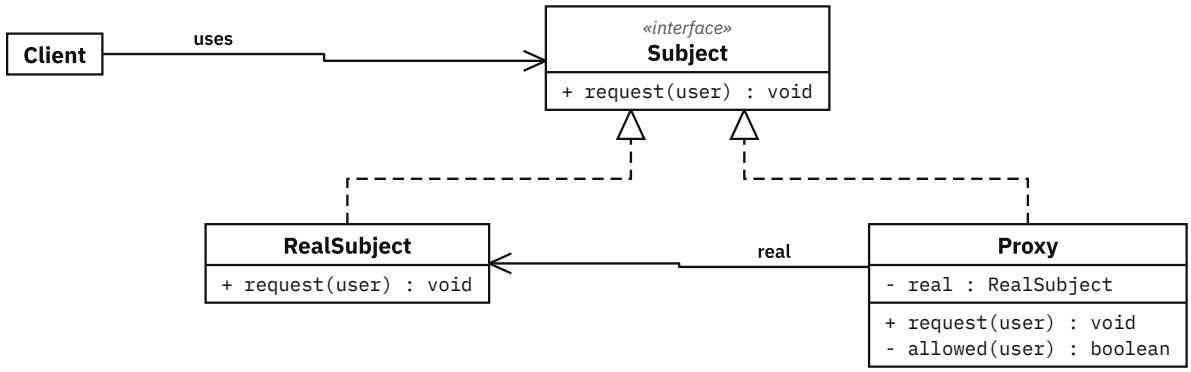
Slot table

example: "only enrolled students may watch recorded lectures; videos load only when played"

ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
Subject «interface»	the thing the user asks for	`VideoLecture` with `play(student)`
RealSubject	`Real` + subject name: the expensive / protected one	`RealVideoLecture`
Proxy	subject name + `Proxy`	`VideoLectureProxy`
the check	"permission / authorised / restricted" → protection; "load only when / large" → virtual (lazy)	`isEnrolled(student)` + create on first play
Client	who asks for the thing	`LmsPlayer`

Skeleton diagram

both implement the subject; the proxy points to the real one




```java
// SUBJECT
public interface Subject {
  void request(String user);
}
// REAL SUBJECT: the real / costly work
public class RealSubject implements Subject {
  public RealSubject() {
    System.out.println("Loading..."); }
  public void request(String user) {
    System.out.println("Serving " + user); }
}
// CLIENT: can't tell it's a proxy
Subject s = new Proxy();
s.request("S001");
```

```java
// PROXY: same interface, check first
public class Proxy implements Subject {
  private RealSubject real;    // null at first

  public void request(String user) {
    if (!allowed(user)) {      // protection
      System.out.println("Denied");
      return; }
    if (real == null)          // virtual (lazy)
      real = new RealSubject();
    real.request(user);        // forward
  }
  private boolean allowed(String user) {
    return user.startsWith("S"); } // STUB
}
```

Check

☐ proxy and real both **implement the same interface** ☐ arrow proxy → real (never the other way) ☐ client holds the **interface** type ☐ named the proxy type (protection / virtual / logging) ☐ check happens **before** the forward

06 Decorator: fill-in kit

Four roles: the general thing, the plain base, the abstract wrapper, and one class per extra.

KIT
DECORATOR

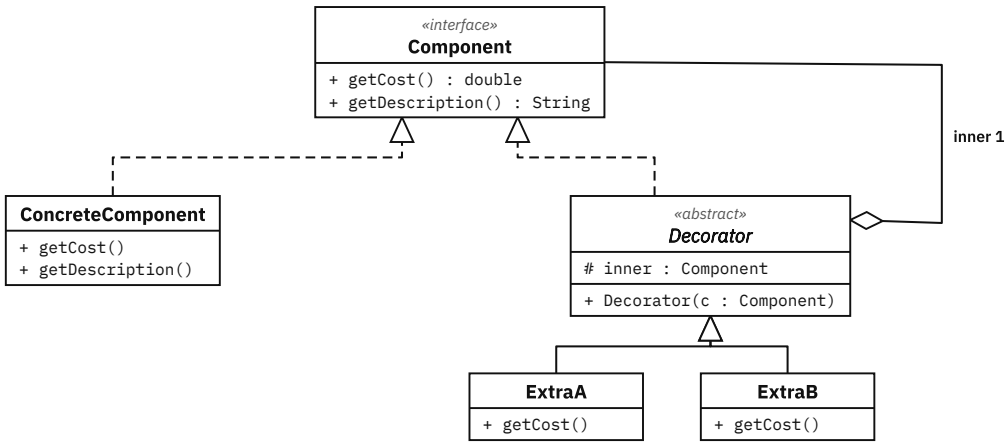
Slot table

example: the burger shop from "Step 1"

ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
Component «interface»	the general thing being priced / extended	`Burger`
operations	what must be calculated ("price", "description", "delivery time"), or the names given	`getCost()` , `getDescription()`
ConcreteComponent	the plain / base one ("basic", "base", "standard")	`BasicBurger`
Decorator «abstract»	general word + `Decorator`	`BurgerDecorator`
ConcreteDecorators	the listed extras / add-ons / toppings / services	`Cheese` , `Bacon` , `LargeSize`

Skeleton diagram

the decorator both implements AND holds a Component: triangle up + diamond arrow




```java
// COMPONENT
public interface Component {
  double getCost();
  String getDescription();
}
// CONCRETE COMPONENT: the plain base
public class ConcreteComponent
    implements Component {
  public double getCost() { return 500; }
  public String getDescription() {
    return "Basic"; }
}
// CLIENT: wrap in any order, any number
Component c = new ConcreteComponent();
c = new ExtraA(c);
c = new ExtraB(c);
c.getCost();   // 500 + 150 + 100 = 750
```

```java
// DECORATOR: is-a Component + has-a Component
public abstract class Decorator
    implements Component {
  protected final Component inner;
  public Decorator(Component c) { inner = c; }
}
// CONCRETE DECORATOR: ask inner, add own part
public class ExtraA extends Decorator {
  public ExtraA(Component c) { super(c); }
  public double getCost() {
    return inner.getCost() + 150; }
  public String getDescription() {
    return inner.getDescription() + ", A"; }
}
// ExtraB: same shape, + 100 and ", B"
```

Check

☐ decorator **implements** Component (dashed triangle) **and** has a Component field (diamond at the decorator) ☐ extras **extend** the decorator (solid triangle) ☐ every method calls `inner.…` then adds ☐ constructor takes the Component ☐ one class per listed extra

07 Template Method: fill-in kit

Write the list of steps from the question. Same-for-everyone steps are written once; "differs" steps are abstract.

KIT
TEMPLATE

Slot table

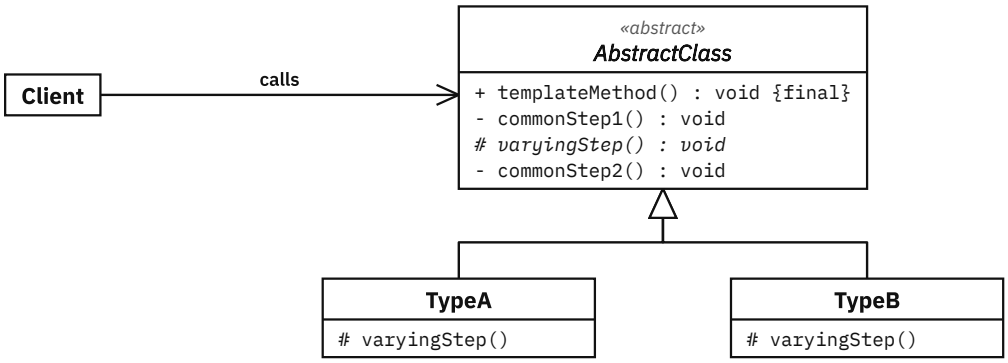
example: loans: verify → assess risk → approve → notify; risk differs per loan type

ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
AbstractClass	the general process / thing all types share	`LoanApplication`
templateMethod() final	the whole process as one verb, or the name given	`processApplication()`
common steps	listed steps that are the **same** for every type	`verifyDocuments()`, `approve()`, `notifyCustomer()`
varying steps (abstract)	steps the question says "differ / vary / are specific"	`assessRisk()`
ConcreteClasses	the listed types, one subclass each	`HomeLoan`, `VehicleLoan`

Trick: write the question's steps as a numbered list. Circle the ones that "differ". Circled = `protected abstract`. The rest = written once. The list order = the order inside the final method.

Skeleton diagram

solid triangles (extends); {final} on the template; italic = abstract




```java
// ABSTRACT CLASS: owns the order
public abstract class AbstractClass {
  public final void templateMethod() {
    commonStep1();      // same for all
    varyingStep();      // each type
    commonStep2();      // same for all
  }
  private void commonStep1() {
    System.out.println("Step 1"); }
  protected abstract void varyingStep();
  private void commonStep2() {
    System.out.println("Step 3"); }
}
```

```java
// CONCRETE CLASS: only the varying step
public class TypeA extends AbstractClass {
  protected void varyingStep() {
    System.out.println("A's way"); }
}
public class TypeB extends AbstractClass {
  protected void varyingStep() {
    System.out.println("B's way"); }
}
// CLIENT: calls only the template method
AbstractClass x = new TypeA();
x.templateMethod();  // Step 1, A's way, Step 3
```

Check

☐ template method **public final** ☐ varying steps **protected abstract** (italic in UML) ☐ subclasses **extend** (solid line), they don't implement ☐ subclasses write only the varying steps ☐ client calls only the template method

08 Chain of Responsibility: fill-in kit

One abstract handler with a "next" link to itself. The levels or checks listed in the question become the handlers, in that order.

KIT
CHAIN

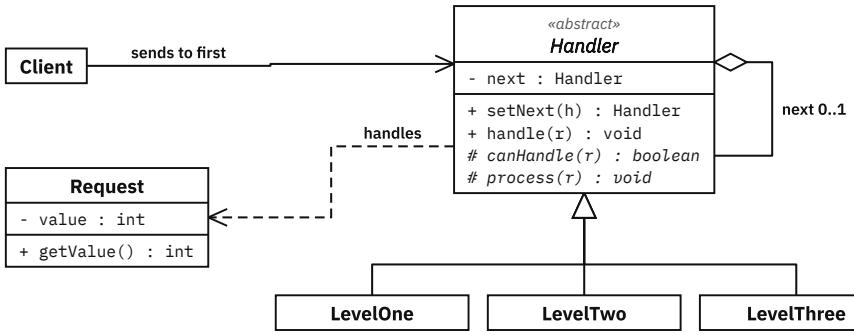
Slot table

example: leave ≤ 2 days supervisor, ≤ 7 manager, longer HR director

ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
Request	the thing being passed along (and the value that decides who handles it)	`LeaveRequest` with `days`
Handler «abstract»	the general word for who deals with it (+ `Handler` if none)	`Approver`
next / setNext()	fixed names	`next`, `setNext()`
ConcreteHandlers	the listed levels / checks / filters, **in the order given**	`Supervisor`, `Manager`, `HRDirector`
handle()	`handle`, or the verb ("approve", "validate", "filter")	`approve(request)`
Client	who sends the request in	`LeavePortal`

Skeleton diagram

the "next" link is ONE loop on the abstract handler, not arrows between concrete handlers




```
// HANDLER
public abstract class Handler {
  private Handler next;
  public Handler setNext(Handler n) {
    next = n; return n; } // lets you chain

  public void handle(Request r) {
    if (canHandle(r)) process(r);      // me
    else if (next != null) next.handle(r);
  }
  protected abstract boolean
      canHandle(Request r);
  protected abstract void process(Request r);
}
// Request: a class with the deciding value,
// e.g. LeaveRequest with an int days field
```

```
// CONCRETE HANDLER (one per level)
public class LevelOne extends Handler {
  protected boolean canHandle(Request r) {
    return r.getValue() <= 2; }
  protected void process(Request r) {
    System.out.println("Level one"); }
}
// LevelTwo: value <= 7 · LevelThree: true

// CLIENT: build once, send to the first
Handler h = new LevelOne();
h.setNext(new LevelTwo())
 .setNext(new LevelThree());
h.handle(new Request(5));  // Level two
```

"Checks in order" questions (validation, filters: every check must pass): make `handle()` return `boolean`: `if (!check(r)) return false; return next == null || next.handle(r);` with one abstract `check(r)`. Levels / approvals / priorities: use the code above.

Check: ☐ `next` field on the abstract handler (self-loop in UML) ☐ `next != null` check ☐ last handler is a fallback ☐ client only calls the first handler ☐ concretes extend (solid triangle)

09 State: fill-in kit

Draw the state diagram first. Its arrows give you the methods; its boxes give you the classes.

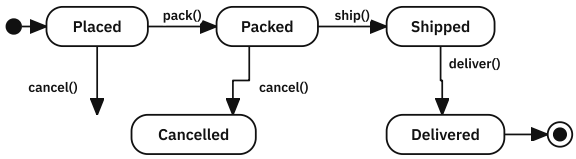
KIT
STATE

Slot table

example: order Placed → Packed → Shipped → Delivered; cancel before shipping

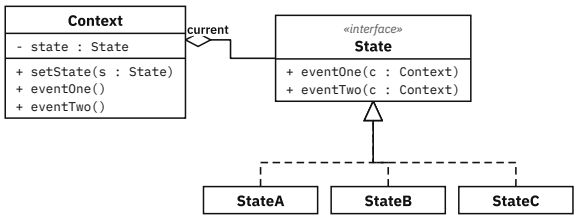
ROLE	FIND IT IN THE QUESTION BY…	EXAMPLE
Context	the thing that "moves through states"	`Order`
State «interface»	context name + `State`	`OrderState`
events = methods	the **label on each arrow** of your state diagram (actions that cause a move)	`pack()`, `ship()`, `deliver()`, `cancel()`
ConcreteStates	each state named in the question + `State`	`PlacedState`, `PackedState`, `ShippedState`, `DeliveredState`, `CancelledState`
setState()	fixed name, on the context	`setState(s)`

1 · State diagram first



Dot = start. Rounded box per state. Every arrow labelled with its event. Bullseye only on a real end state.

2 · Class skeleton




```
// STATE: one method per arrow label
public interface State {
  void eventOne(Context c);
  void eventTwo(Context c);
}
// CONTEXT: holds the current state, delegates
public class Context {
  private State state = new StateA(); // start
  public void setState(State s) { state = s; }
  public void eventOne() {
    state.eventOne(this); }
  public void eventTwo() {
    state.eventTwo(this); }
}
```

```
// CONCRETE STATE: allowed events move,
// the rest are refused
public class StateA implements State {
  public void eventOne(Context c) {
    c.setState(new StateB()); }   // A -> B
  public void eventTwo(Context c) {
    System.out.println("Not allowed in A"); }
}
// StateB, StateC: same shape, following
// the arrows of your state diagram
// CLIENT
Context c = new Context();
c.eventOne();   // now in StateB
```

Check: ☐ every arrow in the state diagram = a method in the interface ☐ every state box = a class ☐ events not on the diagram for that state print "not allowed" ☐ context has `setState()` and **no** `if (state == …)` ☐ diamond at the context, dashed triangles from states to the interface

P Two patterns in one question, and if you freeze

Recent papers put 3–4 patterns into one scenario. Keep each pattern as its own small group, then join them with one line.

PRACTICE
JOINING

Joining patterns: three rules

1. **One slot table per pattern.** Don't mix them. Two patterns = two little tables on your rough page.
2. **Draw each pattern as its own group** (its own skeleton), side by side.
3. **Join with one arrow where one class uses the other**, and say in one line **which method calls which**. In code, the call sits inside the method that needs it.

COMBINATION	THE JOINING ARROW	THE JOINING LINE OF CODE
State + Decorator + Singleton **2025 Q1**	Kiosk (context) ◇→ CheckoutItem; ScanningState ⇢ TaxCalculator	inside `checkout()`: `TaxCalculator.getInstance().calculateTax(k.subtotal())`
Chain + Factory + Template + Singleton **2025 Q4**	OrderService → Validator, → PaymentFactory, ⇢ OrderIdGenerator	`if (chain.validate(o)) PaymentFactory.create(m).handlePayment(o);`
Facade + Adapter **2022 Q1**	a subsystem → its adapter (Target)	inside the subsystem: `provider.search(...)` where `provider` is an adapter
State + Adapter `Quiz 1`	PaymentDueState ⇢ PaymentProcessor (Target)	inside `pay()`: `if (processor.pay(amt)) s.setState(new ClosedState());`
Factory + Singleton `Quiz 2`	Factory ⇢ product; product uses the Singleton connection	`DatabaseFactory.getDatabase(t)`; inside it `ConnectionManager.getInstance()`
Chain + Template `Quiz 7`	none extra: the handler's `handle()` is the final template method	`public final void handle(r) { log(); attemptToSolve(r); log(); … }`

If you freeze: the minimum that still gets most of the marks

Diagram, short on time	Draw only the pattern's core: the interface/abstract box on top, two concrete boxes under it with triangles, the special box with its special link. Correct names and arrows on 4 boxes beat 9 boxes with wrong arrows.
Code, short on time	Write the interface and the pattern class (decorator / adapter / proxy / context / handler). That's where the pattern lives. Then one line: "Other concrete classes follow the same structure."
Can't think of a name	Use the role name + the general word: `BookFactory`, `ScannerAdapter`, `OrderState`. A clear role name is always acceptable.
Lost which box is which	Look at your slot table, not the question. That's why you wrote it first.

P Practice: three new scenarios

Tonight: cover the answers, fill the slot table yourself, then check. Every name comes from the scenario's words or a role suffix.

PRACTICE
SELF-TEST

A · Gym membership

SCENARIO

"A gym sells a basic membership. Members can add optional services: personal trainer, locker, and sauna access. The monthly fee must be calculated for any combination without creating a subclass for each combination."

ROLE	ANSWER
Pattern	Decorator ("optional services", "any combination without subclasses")
Component · operation	`Membership` · `getMonthlyFee()`, `getDescription()`
ConcreteComponent	`BasicMembership`
Decorator · extras	`MembershipDecorator` · `PersonalTrainer`, `Locker`, `SaunaAccess`
Key code line	`return inner.getMonthlyFee() + 3000;` (in `PersonalTrainer`)

B · Traffic light

SCENARIO

"A smart traffic light can be Red, Green or Yellow, and Flashing when it has a fault. A timer event moves Red → Green → Yellow → Red. A fault event from any colour moves it to Flashing; a repair event moves Flashing back to Red."

ROLE	ANSWER
Pattern	State ("can be Red, Green...", "moves to")
Context · State interface	`TrafficLight` · `LightState`
Events = methods	`timer(TrafficLight t)`, `fault(TrafficLight t)`, `repair(TrafficLight t)`
Concrete states	`RedState`, `GreenState`, `YellowState`, `FlashingState`
Key code line	in `GreenState.timer(t)`: `t.setState(new YellowState());` · in `FlashingState.timer(t)`: print "not allowed"

C · Insurance claims

SCENARIO

"Every insurance claim goes through the same process: register the claim, assess the damage, calculate the payout, and notify the customer. Damage assessment and payout calculation differ for vehicle, health and property claims. Claims above Rs 1,000,000 must be approved by a senior officer, smaller ones by a junior officer."

ROLE	ANSWER
Patterns	Template Method (same process, two steps differ) + Chain of Responsibility (approval levels by amount)
Template slots	`InsuranceClaim` · final `processClaim()` · common `registerClaim()`, `notifyCustomer()` · abstract `assessDamage()`, `calculatePayout()` · `VehicleClaim`, `HealthClaim`, `PropertyClaim`
Chain slots	request = the claim's payout amount · `ClaimApprover` with `next` · `JuniorOfficer` (≤ 1,000,000) → `SeniorOfficer` (fallback)
Joining line	inside `processClaim()`, after `calculatePayout()`: `approverChain.handle(this);`

Blank slot sheet

P

Photocopy or print extra copies for practice tonight. In the exam, draw the same little table in your rough-work space.

PRACTICE
WORKSHEET

Question: _____ Pattern: _____ Marks (diagram / code): ____ / ____

ROLE (COPY FROM THE KIT PAGE)	MY WORD FROM THE QUESTION	FIELD / METHOD NAMES

Question: _____ Pattern: _____ Marks (diagram / code): ____ / ____

ROLE (COPY FROM THE KIT PAGE)	MY WORD FROM THE QUESTION	FIELD / METHOD NAMES

Then: ① copy the skeleton diagram with these words · ② write code in table order (interface → one concrete → pattern class → more concretes → client) · ③ tick the checklist on the pattern page.